Revisión del script — Versión 6

Archivo: TERMINAL ZACARIASS VERSION 6 · BD terminal · 24 de octubre de 2026

Veredicto: el script corre completo en PostgreSQL 16 y ya incluye los cambios propuestos en V002 (no hay que aplicar V002 aparte). **El login ya se puede construir.** Para la venta faltan dos ajustes: los datos del titular y los códigos de error del trigger de boletos.

Cómo se revisó

Se creó una base vacía, se ejecutó el script completo, se insertaron datos de prueba y se intentó romper cada regla a propósito.

1. Lo que ya funciona

Prueba	Resultado
El script se ejecuta completo de principio a fin	**Pasa**
Precio por tramo y descuento por tarifa (tablas precio_tramo y tarifa.descuento)	**Pasa**
Venta con recibido, reemplaza_a, cancelada_por y cancelada_en	**Pasa**
Sobreventa del mismo asiento en tramos que se enciman: se rechaza (23P01)	**Pasa**
Conductor sin licencia o con licencia vencida: se rechaza (Z0006)	**Pasa**
Unidad o chofer con dos viajes encimados: se rechaza (Z0012)	**Pasa**
Folio fuera del rango de la taquilla: se rechaza (Z0002)	**Pasa**
Recibido menor que el total: se rechaza	**Pasa**
Fechas con zona horaria (timestamptz) e índice por fecha de salida	**Pasa**
Cuenta con activo y puesto ADMINISTRADOR	**Pasa**

2. Necesario para la venta

2.1 Los datos del titular no caben en ventas.pasajero. La pantalla de venta pide "Nombre completo" y "CURP o identificación". La tabla exige nombre y dos apellidos por separado, y solo acepta CURP. En la prueba:

Caso	Resultado actual
Pasajero con un solo apellido (o "Familia Torres")	**Se rechaza: ap_materno es obligatorio**
Pasajero que se identifica con INE o pasaporte	**Se rechaza: no cumple el formato de CURP**

Propuesta: en la base, ap_materno opcional y una columna nueva identificacion TEXT (INE, pasaporte); curp sigue opcional y con su validación. En el front, la pantalla pedirá nombre, apellido paterno y apellido materno (opcional) por separado.

```
ALTER TABLE ventas.pasajero ALTER COLUMN ap_materno DROP NOT NULL;
ALTER TABLE ventas.pasajero ADD COLUMN identificacion TEXT;
```

2.2 Los códigos de error del trigger de boletos chocan con el contrato. `ventas.fn_preparar_boleto` usa Z0008 a Z0011, que en el contrato de datos ya significan otra cosa. El backend traduciría mal el error y el taquillero vería un mensaje equivocado.

Código	Uso en el trigger	Significado en el contrato	Cambiar a
Z0008	El asiento no existe	RUTA_EN_USO	**Z0013**
Z0009	Origen fuera de la ruta	FOLIOS_EMITIDOS	**Z0014**
Z0010	Destino fuera de la ruta	TAQUILLA_OCUPADA	**Z0015**
Z0011	Destino antes del origen	NO_ELIMINABLE	**Z0016**

2.3 Faltan las funciones y procedures de la venta (no es un error del script, es lo siguiente por construir): buscar_salidas y asientos_ocupados como funciones de solo lectura, y confirmar_venta como PROCEDURE llamado con CALL (por Pgpool, todo lo que escribe va con CALL).

3. Necesario más adelante

Problema	Cuándo se necesita	Propuesta
El despachador y el administrador no tienen terminal: su cuenta no tiene taquilla y la sesión necesita saber en qué terminal trabajan	Pantallas del despachador	`personal_app.id_terminal` (para el taquillero, la de su taquilla)
No se pueden agregar registros nuevos a los catálogos: al insertar la tarifa MAESTRO sale "id 1 ya existe". Los catálogos se llenaron con IDs fijos y el contador quedó en 1	Pantallas del administrador	Un `setval` por catálogo después de sus INSERT

```
SELECT setval(pg_get_serial_sequence('ventas.tarifa', 'id_tarifa'), (SELECT MAX(id_tarifa) FROM ventas.tarifa));
```

4. Orden sugerido

1. Versión 7 con los puntos 2.1 y 2.2.

2. Escribir confirmar_venta, buscar_salidas y asientos_ocupados.

3. Mientras tanto, el backend arranca con el inicio de sesión, que ya no depende de nada.

4. Los puntos de la sección 3 pueden esperar a las pantallas del despachador y del administrador.